

Thread Synchronization 1



Thread communication in a parallel program

➤ Shared- memory

- Multiple threads running on the the same computer
- Mutliple threads use the same memory area where they can read/write values
- Apply in shared-memory systems
- Communication done by using shared-resource
 - Shared variable, shared files, database ...

Thread communication in a parallel program

- Message passing
 - Tasks running on the different computers in a network
 - Apply in distributed-memory systems
 - Communication done by message passing
 - One task sends a message
 - Another task receives message
 - Follow a predefined communication protocol

Incorrect output of a multithread program

- The order of task execution in a concurrent program without synchronization
 - Nondeterministic
- The output of the program is often incorrect

Example of a program with two threads

count :

Thread 1

A light blue square box with a red border containing the code `count++;`

```
count++;
```

Thread 2

A purple square box with a red border containing the code `count --;`

```
count --;
```

Instruction execution

count++;

R1 ← count

R1 ← R1 + 1

count ← R1

Instruction execution

count --;

$R2 \leftarrow \text{count}$

$R2 \leftarrow R2 - 1$

$\text{count} \leftarrow R2$

One possible output

$R1 \leftarrow count$

The value 0 of shared variable count is fetched into register R1.

$R2 \leftarrow count$

The value 0 of count is fetched into register R2.

$R2 \leftarrow R2 - 1$

R2 is decreased by 1 (now -1).

$count \leftarrow R2$

R2 is returned to the shared variable count in the shared memory.

$R1 \leftarrow R1 + 1$

R1 is increased by 1 (now 1).

$count \leftarrow R1$

R1 (now 1) is returned to the shared variable count in the shared memory.

count = 1

Other possible outputs

$R1 \leftarrow count$
 $R1 \leftarrow R1 + 1$
 $count \leftarrow R1$
 $R2 \leftarrow count$
 $R2 \leftarrow R2 - 1$
 $count \leftarrow R2$

count = 0

$R1 \leftarrow count$
 $R2 \leftarrow count$
 $R2 \leftarrow R2 - 1$
 $count \leftarrow R2$
 $R1 \leftarrow R1 + 1$
 $count \leftarrow R1$

count = 1

$R2 \leftarrow count$
 $R1 \leftarrow count$
 $R1 \leftarrow R1 + 1$
 $count \leftarrow R1$
 $R2 \leftarrow R2 - 1$
 $count \leftarrow R2$

count = -1

$R2 \leftarrow count$
 $R2 \leftarrow R2 - 1$
 $count \leftarrow R2$
 $R1 \leftarrow count$
 $R1 \leftarrow R1 + 1$
 $count \leftarrow R1$

count = 0

Race condition

- Multiple threads attempt to access and modify a shared resource simultaneously or concurrently
 - *Read or write the shared resource*
- Race condition can provoke error situations or data **inconsistency**
- The final value of the share resource becomes unpredictable

Data race – the battle of memory control

- Data race is a specific type of race condition
- The **same memory** is shared between several threads. At least one of them changes it (write access) without a proper synchronization mechanism

Solutions: Synchronization mechanisms

- Mutex
- Semaphore
- Monitor
-

Synchronization Mechanism

➤ Objectives

- Solve data race problem in a multithread program

➤ Ideas

- Create a critical section and ensure mutual exclusion in the critical section (common)

Synchronization Mechanism

➤ Objectives

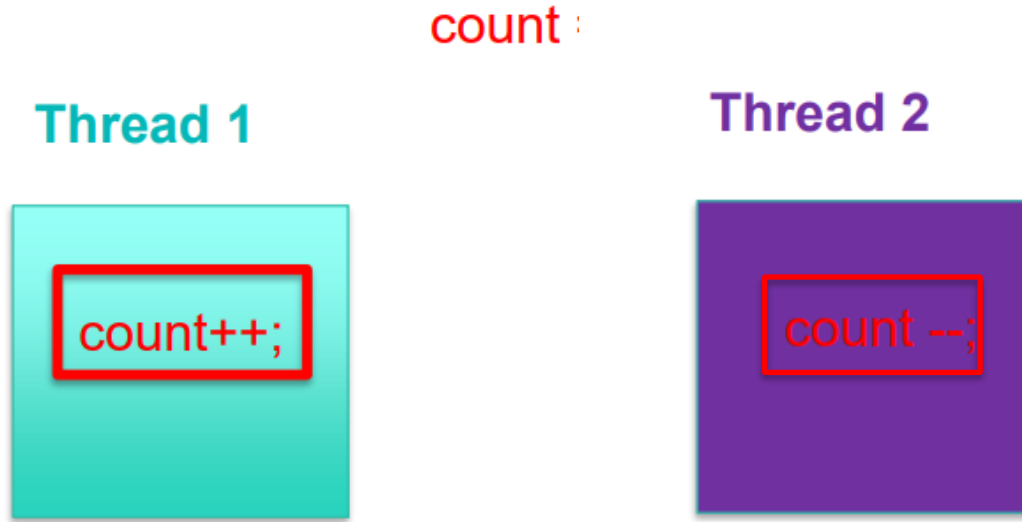
- Solve **data race** problem in a multithread program

➤ Ideas

- Create a **critical section** and ensure **mutual exclusion** in the critical section (common)

Critical section

- Block of code that accesses a shared-resource



Mutual Exclusion

- Mutual Exclusion: **only one thread can access a critical section at any given time**
 - One thread “excludes” all other threads while it executes the critical section
- Mutual Exclusion is the most prevalent form of coordination in a shared-memory system

Synchronization Mechanism

- A synchronization mechanism can introduce significant **overhead** to the program
 - Slow down the execution of the program
 - Consume a large amount of resource
 - Limit scalability when the number of tasks increases
 - ...
- A synchronization mechanism can significantly **degrade** the overall performance

Forster' methodology: Aggregation

- Combine groups of tasks to form fewer but larger tasks, thereby reducing communication requirement (lower communication overhead)
- Increasing the locality of parallel algorithm, lower communication overhead
 - Combine primitive tasks that communicate with each other
 - Eliminate communication because data values for primitive tasks are in memory of consolidated tasks



Synchronization Mechanism

- Big tasks with low intercommunication
 - Reduce overhead
 - Perhaps not benefit from multiple cores (parallel execution)
- Small tasks with high intercommunication
 - High overhead
 - Degrade performance of the program
- Trade-off

Synchronization Mechanism

› Monitor

- › Use `synchronized` keyword to create a critical section
- › Synchronized keyword used with
 - › One or multiple methods
 - › Blocks of code

› Lock

- › Reentrant lock
- › ReadWrite lock

Monitor

Synchronized a method

- Add **synchronized** in the method declaration

```
synchronized void printNumber(int n){  
    ....  
}
```

```
public synchronized void synchronizeSum() {  
    sum ++;  
}
```

Synchronized a method

Critical section



```
class Wallet {  
    3 usages  
    private int balance = 1000;  
    1 usage  
    public synchronized void pay(String user, int amount) {  
        // Some work outside the synchronized block  
        System.out.println(user + " wants to pay " + amount);  
        int fee = 5; // local variable, thread-safe  
        int total = amount + fee; // more local variables  
  
        // Only this part is synchronized  
        int newBalance;  
        if (balance >= total) {  
            balance -= total; // shared state update  
            newBalance = balance; // capture correct value  
        } else {  
            System.out.println("Not enough money for " + user);  
            return;  
        }  
        // More work outside the synchronized block  
        System.out.println(user + " paid " + amount +  
            " (fee " + fee + "), new balance = " + newBalance);  
    }  
}
```

Synchronized a method

```
class Number{
    synchronized void printNumber(int n){
        int temp = 1;
        for(int i=1;i<=10;i++){
            System.out.println(Thread.currentThread().getName() + ": " + n*i);
            temp = n*temp;
            try{
                Thread.sleep(500);
            }catch(Exception e){
                System.out.println(e);
            }
        }
    }
}
```

- The entire method is a critical section
- Only one thread can execute the critical section at a given time

Synchronized several methods

```
public class EventStorage {  
    public synchronized void set(){  
        ...  
    }  
  
    public synchronized void get(){  
        ...  
    }  
}
```

- Several methods become a critical section
- Only one thread can execute the critical section at a time
- A thread can execute only **one method** in the critical section at a given time
- May reduce performance

Synchronized a block of code

- Must pass an **object reference**
 - Use **this** keyword as an object reference
 - Use other object references

Synchronized a block of code – use **this** keyword

Critical section



```
class Wallet {  
    3 usages  
    private int balance = 1000;  
    1 usage  
    public void pay(String user, int amount) {  
        // Some work outside the synchronized block  
        System.out.println(user + " wants to pay " + amount);  
        int fee = 5;           // local variable, thread-safe  
        int total = amount + fee; // more local variables  
  
        // Only this part is synchronized  
        int newBalance;  
        synchronized (this) {  
            if (balance >= total) {  
                balance -= total;           // shared state update  
                newBalance = balance;       // capture correct value  
            } else {  
                System.out.println("Not enough money for " + user);  
                return;  
            }  
        }  
  
        // More work outside the synchronized block  
        System.out.println(user + " paid " + amount +  
            " (fee " + fee + ")", new balance = " + newBalance);  
    }  
}
```

Synchronized a block of code – use other object references

```
private final Object controlCars, controlMotorcycles;

public void carComeIn() {
    synchronized (controlCars) {
        numberCars++;
    }
}

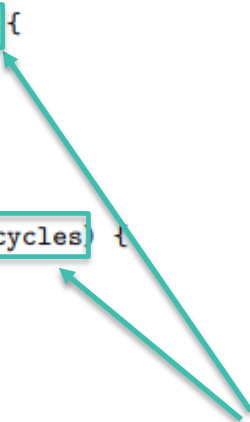
public void motoComeIn() {
    synchronized (controlMotorcycles) {
        numberMotorcycles++;
    }
}
```

Synchronized a block of code – use other object references

```
private final Object controlCars, controlMotorcycles;

public void carComeIn() {
    synchronized (controlCars) {
        numberCars++;
    }
}

public void motoComeIn() {
    synchronized (controlMotorcycles) {
        numberMotorcycles++;
    }
}
```



- Object references

- Two objects: **controlCars** and **controlMotorcycles**
- **controlCars** object synchronizes the access to the **numberCars** attribute
- **controlMotorcycles** object synchronizes the access to the **numberMotorcycles** attribute

Synchronized a block of code – use other object references

```
private final Object controlCars, controlMotorcycles;

public void carComeIn() {
    synchronized (controlCars) {
        numberCars++;
    }
}

public void motoComeIn() {
    synchronized (controlMotorcycles) {
        numberMotorcycles++;
    }
}
```

Thread 1

Thread 2

- Two blocks can be executed by **Thread 1** and **Thread 2** at the same time
- But each block is executed by only one thread at a given time

Synchronized a block of code – use other object references

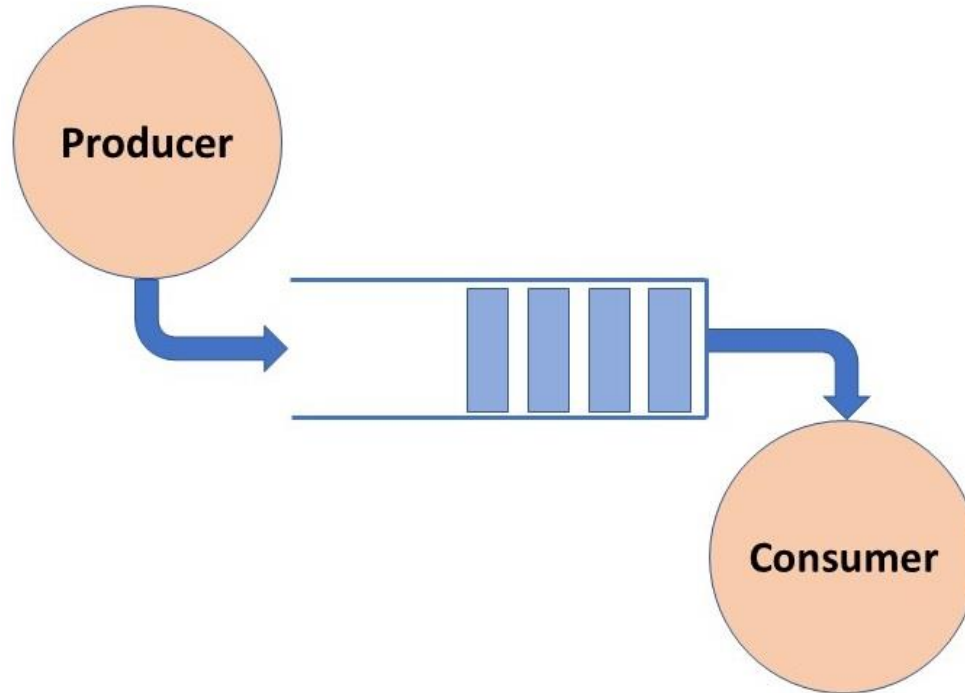
```
private final Object controlCars, controlMotorcycles;

public void carComeIn() {
    synchronized (controlCars) {
        numberCars++;
    }
}

public void motoComeIn() {
    synchronized (controlMotorcycles) {
        numberMotorcycles++;
    }
}
```

- **More flexible**
- There are different critical sections
- Each critical section can be executed by a thread at a time
- Enhance performance

Using condition in a synchronized code



Using condition in a synchronized code - Producer

```
public synchronized void put(){
    while (storage.size()==maxSize){ // When the queue is full, the producer stops putting an
        item into the queue
        try {
            wait(); // Stop by sleeping
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
    }
    storage.add(new Date());
    System.out.printf("Put: %d\n",storage.size());
    notify(); // Wakes up when there is a room in the queue
}
```

Using condition in a synchronized code - Producer

```
public synchronized void put(){
    while (storage.size()==maxSize){ // When the queue is full, the producer stops putting an
        item into the queue
        try {
            wait(); // Stop by sleeping
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
    }
    storage.add(new Date());
    System.out.printf("Put: %d\n",storage.size());
    notify(); // Wakes up when there is a room in the queue
}
```

Using condition in a synchronized code - Consumer

```
public synchronized void remove(){  
    while (storage.size()==0) { // When the queue is full, the producer stops removing an item from the queue  
        try {  
            wait(); // Stop by sleeping  
        } catch (InterruptedException e) {  
            e.printStackTrace();  
        }  
    }  
    String item=storage.poll().toString();  
    System.out.printf("Remove: %d: %s\n",storage.size(),item);  
    notify(); // Wakes up when there is an item space in the queue  
}
```



Process data

Lock

Lock

- Implements the critical section
- Allows mutual exclusion
- More flexible than Monitor

Reentrant Lock

- Implementation based on Lock interface
 - `java.util.concurrent.locks` package
- `ReentrantLock` class
 - `lock();`
 - `unlock();`

Reentrant Lock

```
public class Task implements Runnable {  
  
    private static ReentrantLock lock = new ReentrantLock();  
    private String name;  
  
    public Task(String name) {  
        this.name = name;  
    }  
  
    public void run() {  
        lock.lock();  
        try {  
            System.out.println( name + ": Running the task");  
            Work.doTask();  
            System.out.println("Task: " + name + ": The execution has finished")  
        } finally {  
            lock.unlock();  
        }  
    }  
}
```

Critical Section



Implementation

```
.....  
private static ReentrantLock lock = new ReentrantLock();  
  
lock.lock();  
try {  
    // BLOCK OF CODE IN CRITICAL SECTION  
} finally {  
    lock.unlock();  
}
```

- Only one thread can hold a lock at a given time
- A thread acquires a lock when it enters critical section
- The thread releases the lock when it leaves the critical section
- Deadlock if lock is not released

ReadWrite Lock

- Enhance concurrency of a program
- ReadWriteLock interface
- ReentrantReadWriteLock class
- Two locks
 - Read operation
 - Write operation

ReadWrite Lock

- Multiple threads can perform read operation simultaneously
- Only one thread can do write operation at a given time
- When one thread is doing write operation, other threads will be blocked

ReadWrite Lock

```
public double getPrice1() {  
    lock.readLock().lock();  
    double value=price1;  
    lock.readLock().unlock();  
    return value;  
}  
  
/**  
 * Establish the prices  
 * @param price1 The price of the first product  
 * @param price2 The price of the second product  
 */  
public void setPrices(double price1, double price2) {  
    lock.writeLock().lock();  
    System.out.printf("%s: PricesInfo: Write Lock Acquired.\n", new Date());  
    try {  
        TimeUnit.SECONDS.sleep(10);  
    } catch (InterruptedException e) {  
        e.printStackTrace();  
    }  
    this.price1=price1;  
    this.price2=price2;  
    System.out.printf("%s: PricesInfo: Write Lock Released.\n", new Date());  
    lock.writeLock().unlock();  
}
```

References

- Maurice Herlihy & Nir Shavit, “The Art of Multiprocessor Programming”, Elsevier, 2008.
- David A Patterson and John L. Hennessy, Computer organization and design : the hardware/software interface, 5th Edition, 2014
- Gonzalez, “Javier Fernandez. Java 9 Concurrency Cookbook”, Packt Publishing Ltd, 2017.